Implementations for parallel problem.

I have included screenshots & verification in the term proj, alongside the repo.

md5

```
#include <stdint.h>
#include <cuda_runtime.h>

#define MD5_F(x, y, z) (((x) & (y)) | (~(x) & (z)))
#define MD5_G(x, y, z) (((x) & (z)) | ((y) & ~(z)))
#define MD5_H(x, y, z) ((x) ^ (y) ^ (z))
#define MD5_I(x, y, z) ((y) ^ ((x) | ~(z)))
#define MD5_ROTL(x, n) (((x) << (n)) | ((x) >> (32 - (n))))

#define MD5_STEP(f, a, b, c, d, x, s, ac) do { \
    (a) += f((b), (c), (d)) + (x) + (uint32_t)(ac); \
    (a) = MD5_ROTL((a), (s)); \
    (a) += (b); \
} while (0)

static __device__ __forceinline__ void md5_transform(uint32_t state[4],
                                                      const uint8_t
block[64])
{
    uint32_t a = state[0], b = state[1], c = state[2], d = state[3];
    uint32_t x[16];

#pragma unroll
    for (int i = 0; i < 16; ++i) {
        int j = i << 2;
        x[i] = (uint32_t)block[j] |
               ((uint32_t)block[j + 1] << 8) |
               ((uint32_t)block[j + 2] << 16) |
               ((uint32_t)block[j + 3] << 24);
    }

    MD5_STEP(MD5_F, a, b, c, d, x[ 0],  7, 0xd76aa478);
    MD5_STEP(MD5_F, d, a, b, c, x[ 1], 12, 0xe8c7b756);
    MD5_STEP(MD5_F, c, d, a, b, x[ 2], 17, 0x242070db);
    MD5_STEP(MD5_F, b, c, d, a, x[ 3], 22, 0xc1bdceee);
    MD5_STEP(MD5_F, a, b, c, d, x[ 4],  7, 0xf57c0faf);
    MD5_STEP(MD5_F, d, a, b, c, x[ 5], 12, 0x4787c62a);
    MD5_STEP(MD5_F, c, d, a, b, x[ 6], 17, 0xa8304613);
    MD5_STEP(MD5_F, b, c, d, a, x[ 7], 22, 0xfd469501);
    MD5_STEP(MD5_F, a, b, c, d, x[ 8],  7, 0x698098d8);
    MD5_STEP(MD5_F, d, a, b, c, x[ 9], 12, 0x8b44f7af);
    MD5_STEP(MD5_F, c, d, a, b, x[10], 17, 0xffff5bb1);
    MD5_STEP(MD5_F, b, c, d, a, x[11], 22, 0x895cd7be);
    MD5_STEP(MD5_F, a, b, c, d, x[12],  7, 0x6b901122);
    MD5_STEP(MD5_F, d, a, b, c, x[13], 12, 0xfd987193);
    MD5_STEP(MD5_F, c, d, a, b, x[14], 17, 0xa679438e);
```

```
        MD5_STEP(MD5_F, b, c, d, a, x[15], 22, 0x49b40821);

        MD5_STEP(MD5_G, a, b, c, d, x[ 1],  5, 0xf61e2562);
        MD5_STEP(MD5_G, d, a, b, c, x[ 6],  9, 0xc040b340);
        MD5_STEP(MD5_G, c, d, a, b, x[11], 14, 0x265e5a51);
        MD5_STEP(MD5_G, b, c, d, a, x[ 0], 20, 0xe9b6c7aa);
        MD5_STEP(MD5_G, a, b, c, d, x[ 5],  5, 0xd62f105d);
        MD5_STEP(MD5_G, d, a, b, c, x[10],  9, 0x02441453);
        MD5_STEP(MD5_G, c, d, a, b, x[15], 14, 0xd8a1e681);
        MD5_STEP(MD5_G, b, c, d, a, x[ 4], 20, 0xe7d3fbc8);
        MD5_STEP(MD5_G, a, b, c, d, x[ 9],  5, 0x21e1cde6);
        MD5_STEP(MD5_G, d, a, b, c, x[14],  9, 0xc33707d6);
        MD5_STEP(MD5_G, c, d, a, b, x[ 3], 14, 0xf4d50d87);
        MD5_STEP(MD5_G, b, c, d, a, x[ 8], 20, 0x455a14ed);
        MD5_STEP(MD5_G, a, b, c, d, x[13],  5, 0xa9e3e905);
        MD5_STEP(MD5_G, d, a, b, c, x[ 2],  9, 0xfcefa3f8);
        MD5_STEP(MD5_G, c, d, a, b, x[ 7], 14, 0x676f02d9);
        MD5_STEP(MD5_G, b, c, d, a, x[12], 20, 0x8d2a4c8a);

        MD5_STEP(MD5_H, a, b, c, d, x[ 5],  4, 0xfffa3942);
        MD5_STEP(MD5_H, d, a, b, c, x[ 8], 11, 0x8771f681);
        MD5_STEP(MD5_H, c, d, a, b, x[11], 16, 0x6d9d6122);
        MD5_STEP(MD5_H, b, c, d, a, x[14], 23, 0xfde5380c);
        MD5_STEP(MD5_H, a, b, c, d, x[ 1],  4, 0xa4beea44);
        MD5_STEP(MD5_H, d, a, b, c, x[ 4], 11, 0x4bdecfa9);
        MD5_STEP(MD5_H, c, d, a, b, x[ 7], 16, 0xf6bb4b60);
        MD5_STEP(MD5_H, b, c, d, a, x[10], 23, 0xbebfbc70);
        MD5_STEP(MD5_H, a, b, c, d, x[13],  4, 0x289b7ec6);
        MD5_STEP(MD5_H, d, a, b, c, x[ 0], 11, 0xeaa127fa);
        MD5_STEP(MD5_H, c, d, a, b, x[ 3], 16, 0xd4ef3085);
        MD5_STEP(MD5_H, b, c, d, a, x[ 6], 23, 0x04881d05);
        MD5_STEP(MD5_H, a, b, c, d, x[ 9],  4, 0xd9d4d039);
        MD5_STEP(MD5_H, d, a, b, c, x[12], 11, 0xe6db99e5);
        MD5_STEP(MD5_H, c, d, a, b, x[15], 16, 0x1fa27cf8);
        MD5_STEP(MD5_H, b, c, d, a, x[ 2], 23, 0xc4ac5665);

        MD5_STEP(MD5_I, a, b, c, d, x[ 0],  6, 0xf4292244);
        MD5_STEP(MD5_I, d, a, b, c, x[ 7], 10, 0x432aff97);
        MD5_STEP(MD5_I, c, d, a, b, x[14], 15, 0xab9423a7);
        MD5_STEP(MD5_I, b, c, d, a, x[ 5], 21, 0xfc93a039);
        MD5_STEP(MD5_I, a, b, c, d, x[12],  6, 0x655b59c3);
        MD5_STEP(MD5_I, d, a, b, c, x[ 3], 10, 0x8f0ccc92);
        MD5_STEP(MD5_I, c, d, a, b, x[10], 15, 0xffeff47d);
        MD5_STEP(MD5_I, b, c, d, a, x[ 1], 21, 0x85845dd1);
        MD5_STEP(MD5_I, a, b, c, d, x[ 8],  6, 0x6fa87e4f);
        MD5_STEP(MD5_I, d, a, b, c, x[15], 10, 0xfe2ce6e0);
        MD5_STEP(MD5_I, c, d, a, b, x[ 6], 15, 0xa3014314);
        MD5_STEP(MD5_I, b, c, d, a, x[13], 21, 0x4e0811a1);
        MD5_STEP(MD5_I, a, b, c, d, x[ 4],  6, 0xf7537e82);
        MD5_STEP(MD5_I, d, a, b, c, x[11], 10, 0xbd3af235);
        MD5_STEP(MD5_I, c, d, a, b, x[ 2], 15, 0x2ad7d2bb);
        MD5_STEP(MD5_I, b, c, d, a, x[ 9], 21, 0xeb86d391);

        state[0] += a;
```

```
    state[1] += b;
    state[2] += c;
    state[3] += d;
}

static __device__ __forceinline__ void md5_hash(const uint8_t *input,
                                                uint32_t len,
                                                uint8_t digest[16])
{
    uint32_t state[4] = {
        0x67452301U, 0xefcdab89U, 0x98badcfeU, 0x10325476U
    };
    uint8_t block[64];
    uint32_t offset = 0;

    while (len - offset >= 64U) {
        md5_transform(state, input + offset);
        offset += 64U;
    }

#pragma unroll
    for (int i = 0; i < 64; ++i) {
        block[i] = 0;
    }

    uint32_t rem = len - offset;
    for (uint32_t i = 0; i < rem; ++i) {
        block[i] = input[offset + i];
    }
    block[rem] = 0x80U;

    if (rem >= 56U) {
        md5_transform(state, block);
#pragma unroll
        for (int i = 0; i < 64; ++i) {
            block[i] = 0;
        }
    }

    uint64_t bits = (uint64_t)len << 3;
#pragma unroll
    for (int i = 0; i < 8; ++i) {
        block[56 + i] = (uint8_t)(bits >> (i << 3));
    }

    md5_transform(state, block);

#pragma unroll
    for (int i = 0; i < 4; ++i) {
        digest[(i << 2)] = (uint8_t)(state[i]);
        digest[(i << 2) + 1] = (uint8_t)(state[i] >> 8);
        digest[(i << 2) + 2] = (uint8_t)(state[i] >> 16);
        digest[(i << 2) + 3] = (uint8_t)(state[i] >> 24);
    }
```

```
}

#undef MD5_STEP
#undef MD5_ROTL
#undef MD5_I
#undef MD5_H
#undef MD5_G
#undef MD5_F
```

bruteforce

```
/*
 * md5_brute.cu  –  GPU brute-force MD5 preimage search.
 *
 * Divides the candidate space [0, |charset|^length) among CUDA threads.
 * Each thread converts its index to a base-|charset| string, computes MD5,
 * and compares with the target.
 *
 * Compile:  nvcc -O2 -o md5_brute md5_brute.cu
 * Usage:    ./md5_brute <hex_md5_target> <charset> <length>
 * Example:  ./md5_brute 5f4dcc3b5aa765d61d8327deb882cf99
abcdefghijklmnopqrstuvwxyz 8
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include "md5.h"
#include <cuda_runtime.h>
#include "md5.cu"

#define MAX_LEN    16
#define MAX_CSET   96
#define BLOCK_SIZE 256

/* ── device data ── */
__constant__ uint8_t d_target[16];
__constant__ char    d_charset[MAX_CSET + 1];
__constant__ int     d_clen;
__constant__ int     d_pwlen;

/* ── kernel ── */
__global__ void brute_kernel(uint64_t base, uint64_t total,
                             int *out_found, uint64_t *out_idx)
{
    if (*out_found) return;
    uint64_t idx = base + blockIdx.x * (uint64_t)blockDim.x + threadIdx.x;
    if (idx >= base + total) return;

    /* convert index to string (base d_clen, little-endian digits) */
    char cand[MAX_LEN + 1];
```

```
    uint64_t tmp = idx;
    for (int i = 0; i < d_pwlen; i++) {
        cand[i] = d_charset[tmp % (uint64_t)d_clen]; // candidate
conversion
        tmp /= (uint64_t)d_clen;
    }
    cand[d_pwlen] = '\0';

    uint8_t digest[16];
    md5_hash((const uint8_t *)cand, (uint32_t)d_pwlen, digest);

    int match = 1;
    for (int i = 0; i < 16; i++) if (digest[i] != d_target[i]) { match=0;
break; }
    if (match) {
        if (atomicExch(out_found, 1) == 0) { // exchange match index
            *out_idx = idx;
        }
    }
}

/* ── helpers ── */
static int hex_to_bytes(const char *hex, uint8_t *out, int n) {
    for (int i = 0; i < n; i++) {
        unsigned v;
        if (sscanf(hex + i*2, "%02x", &v) != 1) return -1; // could replace
scanf with custom conversion for speed?
        out[i] = (uint8_t)v;
    }
    return 0;
}

static uint64_t ipow64(uint64_t base, int exp) {
    uint64_t r = 1;
    for (int i = 0; i < exp; i++) r *= base;
    return r;
}

int main(int argc, char **argv) {
    if (argc != 4) { fprintf(stderr,"usage: %s <md5_hex> <charset>
<length>\n",argv[0]); return 1; } // validate inputs

    uint8_t target[16];
    if (strlen(argv[1]) != 32 || hex_to_bytes(argv[1], target, 16) < 0) {
// validate input is a hash
        fprintf(stderr,"bad md5 hex\n"); return 1;
    }
    const char *charset = argv[2];
    int clen = (int)strlen(charset);
    int pwlen = atoi(argv[3]);
    if (clen < 1 || clen > MAX_CSET || pwlen < 1 || pwlen > MAX_LEN) {
        fprintf(stderr,"charset len 1-%d, password len 1-%d\n", MAX_CSET,
MAX_LEN); return 1;
    }
```

```
    uint64_t space = ipow64((uint64_t)clen, pwlen); // calc search space
size
    printf("Search space: %llu candidates  charset_len=%d  pwlen=%d\n",
           (unsigned long long)space, clen, pwlen);

    cudaMemcpyToSymbol(d_target,  target,  16);
    cudaMemcpyToSymbol(d_charset, charset, (size_t)(clen+1));
    cudaMemcpyToSymbol(d_clen,    &clen,   sizeof(int));
    cudaMemcpyToSymbol(d_pwlen,   &pwlen,  sizeof(int));

    int *d_found; cudaMalloc(&d_found,  sizeof(int));
cudaMemset(d_found, 0, sizeof(int));
    uint64_t *d_idx;   cudaMalloc(&d_idx,    sizeof(uint64_t));
cudaMemset(d_idx, 0, 8);

    const uint64_t BATCH = (uint64_t)BLOCK_SIZE * 65536; /* tune as needed,
maximizes saturation */
    int host_found = 0;
    uint64_t hashes_done = 0;
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);
    cudaEventRecord(start);

    for (uint64_t base = 0; base < space && !host_found; base += BATCH) {
        uint64_t todo  = (base + BATCH < space) ? BATCH : (space - base);
        uint32_t grids = (uint32_t)((todo + BLOCK_SIZE - 1) / BLOCK_SIZE);
        brute_kernel<<<grids, BLOCK_SIZE>>>(base, todo, d_found, d_idx);
// dispatch kernel
        cudaDeviceSynchronize();
        hashes_done += todo;
        cudaMemcpy(&host_found, d_found, sizeof(int),
cudaMemcpyDeviceToHost);
        if (base % (BATCH * 16) == 0)
            printf("  Progress: %llu / %llu\n", (unsigned long long)base,
                   (unsigned long long)space);
    }

    cudaEventRecord(stop);
    cudaEventSynchronize(stop);
    float elapsed_ms = 0.0f;
    cudaEventElapsedTime(&elapsed_ms, start, stop);

    if (host_found) {
        uint64_t idx;
        cudaMemcpy(&idx, d_idx, sizeof(uint64_t), cudaMemcpyDeviceToHost);
        /* recover string on CPU */
        char cand[MAX_LEN + 1]; uint64_t tmp = idx;
        for (int i = 0; i < pwlen; i++) { cand[i]=charset[tmp%clen];
tmp/=clen; } // conversion to base_k
        cand[pwlen]='\0';
        printf("FOUND: \"%s\"  (index %llu)\n", cand, (unsigned long
long)idx);
```

```
    } else {
        printf("Not found in search space.\n");
    }
    double elapsed_s = (double)elapsed_ms / 1000.0;
    double hashes_per_second = elapsed_s > 0.0 ? (double)hashes_done /
elapsed_s : 0.0;
    printf("Hashes/sec: %.2f  (%llu hashes in %.3f seconds)\n",
           hashes_per_second, (unsigned long long)hashes_done, elapsed_s);

    cudaEventDestroy(start); cudaEventDestroy(stop);
    cudaFree(d_found); cudaFree(d_idx);
    return 0;
}
```

Output

```
bash-5.1$ ./md5_brute_gpu 8200fdece927b56ffcfeae7518d5df61 abcsquirdw12 9
Search space: 5159780352 candidates  charset_len=12  pwlen=9
  Progress: 0 / 5159780352
  Progress: 268435456 / 5159780352
  Progress: 536870912 / 5159780352
  Progress: 805306368 / 5159780352
  Progress: 1073741824 / 5159780352
  Progress: 1342177280 / 5159780352
  Progress: 1610612736 / 5159780352
  Progress: 1879048192 / 5159780352
  Progress: 2147483648 / 5159780352
  Progress: 2415919104 / 5159780352
  Progress: 2684354560 / 5159780352
  Progress: 2952790016 / 5159780352
  Progress: 3221225472 / 5159780352
  Progress: 3489660928 / 5159780352
FOUND: "squidward"  (index 3693092739)
Hashes/sec: 7153632472.24  (3707764736 hashes in 0.518 seconds)
```